Major Project I - Report

Zigbee based Machine to Machine Communication for On-road Low-visibility Collision Prevention

Submitted by

Nitesh Bhardwaj (21104023)

Ashutosh Taneja (21104033)

Tanveer Singh (21104061)

Shivam Dhiman (21104062)

Amandeep Singh (21104065)

Under the guidance of

Prof. Dr Balwinder Singh

Submitted to

Department of Electrical Engineering

Punjab Engineering College (Deemed to be University), Chandigarh, India

DECLARATION

We, hereby, declare that the report entitled **"Zigbee based Machine to Machine Communication for on-road Low Visibility Collision Prevention"** is an authentic record of our work at Punjab Engineering College as part of the requirements for the course **Major Project I**. This work has been conducted under the guidance of **Dr. Balwinder Singh from August 2024 to December 2024.**

Dated: 1 December 2024

Nitesh Bhardwaj (21104023)

Ashutosh Taneja (21104033)

Tanveer Singh (21104061)

Shivam Dhiman (21104062)

Amandeep Singh (21104065)

It is certified that the above statement made by the student is correct to the best of our knowledge and belief.

Prof. Dr. Balwinder Singh Surjan

(Professor)

(Electrical Engineering Department)

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to all those who have supported us in preparing this project report.

First and foremost, we sincerely thank our mentor, Dr. Balwinder Singh, for his invaluable guidance, constructive feedback, and constant encouragement, which have been instrumental in shaping the direction of this project.

We are also profoundly grateful to *Punjab Engineering College*, particularly the Electrical Engineering Department, for providing the resources and a conducive environment essential for our research.

Our appreciation extends to the faculty members whose teachings and advice have greatly enriched our understanding and approach throughout this project.

Finally, we acknowledge the teamwork and collaboration among all group members, whose dedication and efforts have made the research process a highly productive and rewarding experience.

TABLE OF CONTENT

DECLARATION **2**
ACKNOWLEDGMENT **3**
TABLE OF CONTENT **4**
ABSTRACT **5**
OBJECTIVE **5**
INTRODUCTION **6**
Why Use Zigbee? 7
LITERATURE SURVEY **9**
EQUIPMENT OVERVIEW **12**
3.1 Zigbee Communication Unit: ESP32-H2 12
3.2 Data Acquisition and Processing Unit: ESP32 13
3.3 GPS Module: Neo-6M 14
3.3 Accelerometer: MPU-6050 15
3.4 Zigbee Mesh Network 16
METHODOLOGY **17**
4.1 Sensor Selection and Calibration 17
4.2 Data Acquisition and Processing 19
4.3 Zigbee Transmission and Network 20
IMPLEMENTATION **23**
5.1 Vehicular node design 23
5.2 GPS and ESP32 Connection 24
5.3 MPU and ESP32 Connection 25
5.4 ESP32 and ESP32H2 Connection (Master-slave) 27
5.5 Zigbee Transmission Architecture 28
EXPERIMENTATION RESULTS **32**
6.1 Packet Delivery Testing 32
Summary of Experimental Results 33
OUTCOMES **34**
7.1 Successful Setup of Zigbee Communication 34
7.2 Integration of Sensors 34
7.3 Hierarchical Communication Structure 34
7.4. Real-Time Data Exchange 35
7.5. Low-Power, Robust Communication 35
CONCLUSION **36**
Real-World Relevance: 37
REFERENCES **38**

ABSTRACT

This project introduces a Zigbee-based Vehicle-to-Vehicle (V2V) communication system to improve road safety in low-visibility conditions like fog and heavy rain. Utilizing ESP32-H2 microcontrollers, it facilitates real-time data exchange of GPS and acceleration. Leveraging Zigbee's low power consumption, mesh networking, and robustness, the system ensures reliable communication, timely collision alerts, and enhanced situational awareness, supporting diverse vehicle types and traffic scenarios.

OBJECTIVE

This project aims to design and develop a reliable **vehicle-to-vehicle (V2V) communication system** utilizing Zigbee technology to enhance road safety, particularly in low-visibility conditions such as pitch-dark nights, dense fog, and heavy rain. The system aims to facilitate real-time data exchange between vehicles, enabling them to communicate critical information such as their location, speed, and direction. By leveraging the low-power, high-reliability features of Zigbee, the system will detect potential collision risks and promptly alert drivers, thereby reducing the likelihood of accidents.

This solution addresses the challenges posed by limited driver visibility and delayed reaction times in adverse weather or lighting conditions. The project emphasizes scalability, ensuring the system can adapt to diverse vehicle types and traffic scenarios while maintaining efficient operation within Zigbee's range and bandwidth constraints. Ultimately, the project aspires to contribute to more intelligent, safer roadways and lay the groundwork for integrating V2V communication with broader intelligent transportation systems (ITS).

INTRODUCTION

In scenarios such as a blizzard, heavy fog, or total darkness, highly reliable communication is required in all cases of navigation, surveillance, and safety applications. High-power wireless systems and visual signs often fail in such harsh environmental conditions as they depend more upon clear visibility or much power utilization. Zigbee, a low-power wireless communication protocol, can be a revolutionary idea. These communication systems show high interferer resistance and low energy consumption, thus suitable for vehicular communication in a low-visibility environment.

For instance, GPS coordinates and acceleration measurements can be shared among the cars as they drive through a snowstorm or thick fog. Such systems also allow monitoring of potential threats to achieve better navigation. Moreover, mesh networking supports Zigbee technology to a great extent in increasing reliability, such as rerouting data when some of the direct links have been compromised.

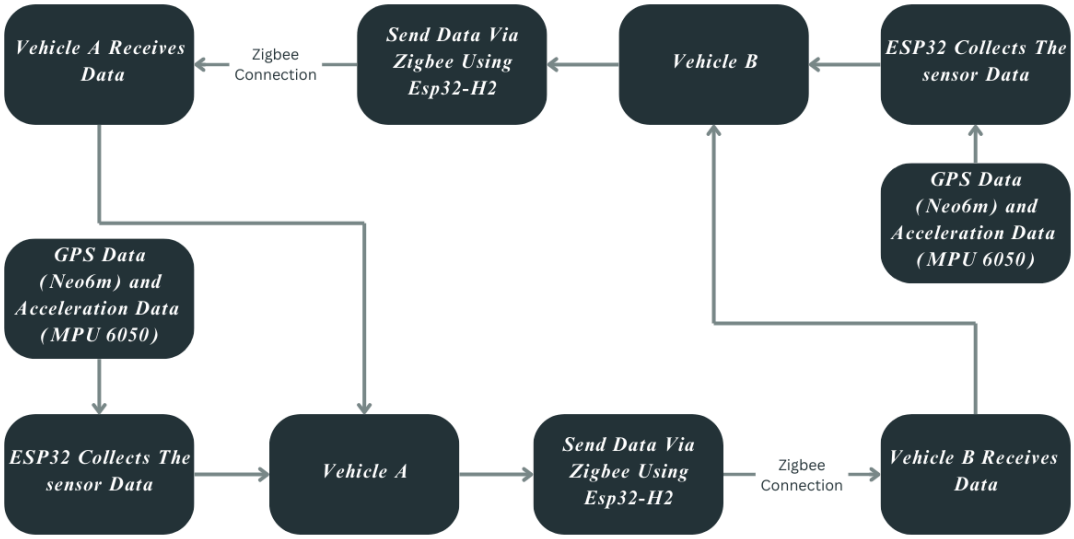

Fig.1 Flowchart for working of V2V communicaton

The project focuses on implementing a Zigbee-based communication system using ESP32-H2 microcontrollers to enable real-time data exchange between two vehicles. Each vehicle has a

GPS module (Neo-6M) for location tracking and an accelerometer (MPU-6050) for motion monitoring. The ESP32-H2 collects and transmits this sensor data to the other vehicle via Zigbee, ensuring seamless bidirectional communication.

This system is designed for low-visibility conditions, such as snowstorms or dense smog, where traditional navigation aids fail. Zigbee's low power consumption and mesh networking capabilities ensure reliable, energy-efficient communication even in challenging environments. The shared GPS and accelerometer data enhance situational awareness, enabling safer navigation, collision avoidance, and dynamic path adjustments.

Why Use Zigbee?

When compared to alternatives like Wi-Fi or Bluetooth, Zigbee offers distinct advantages that make it the preferred choice for this use case:

1) **Low Power Consumption**:
    a) Zigbee is specifically designed for low-power devices, allowing battery-powered units to operate for extended periods.
    b) This is critical for vehicles operating in remote or low-maintenance areas.
2) **Mesh Networking Capability**:
    a) Zigbee supports a self-healing mesh network, enabling devices to relay data through intermediate nodes.
    b) This ensures reliable communication even if some nodes are out of range or blocked due to environmental conditions like snow or smog.
3) **Scalability**:
    a) Zigbee networks can support up to 65,000 nodes, making them suitable for large-scale deployments in IoT systems.
4) **Cost-Effective**:
    a) Zigbee modules are relatively inexpensive compared to Wi-Fi solutions.
    b) They require less infrastructure and energy, reducing the overall cost of deployment and maintenance.

5) **Robustness in Harsh Conditions**:
    a) With strong interference resistance and low bandwidth requirements, Zigbee performs reliably in environments where Wi-Fi may struggle.
6) **Compact Data Transmission**:
    a) Zigbee is optimized for transmitting small packets of data, such as sensor readings, making it an efficient choice for IoT applications.

LITERATURE SURVEY

1. Machine-to-Machine Communication: An Overview of Opportunities, Computer Networks, Volume 145, 2018, Pages 255-276, ISSN 1389-1286

   By: Oluwatosin Ahmed Amodu, Mohamed Othman,

Machine-to-machine (M2M) communication refers to the autonomous interaction between many devices to carry out tasks such as sensing, processing, and actuation without the need for human intervention. These devices include various technologies, such as meters in intelligent grids, electronics, servers, and navigation sensors, which relay information across networks. A key distinction of M2M communication is its ability to function without human supervision, enabling seamless communication among smart devices. This capability is increasingly important in modern applications where real-time data exchange and automated responses are essential.

2. Study on ZigBee technology *2011 3rd International Conference on Electronics Computer Technology*, Kanyakumari, India, 2011, pp. 297-301

   By: C. M. Ramya, M. Shanmugaraj and R. Prabakaran

ZigBee is a network protocol developed by the ZigBee Alliance, which includes various companies from the semiconductor and software industries. ZigBee, distinct from IEEE 802.15.4, is a standard that operates on the IEEE 802.15.4 hardware. While the IEEE 802.15.4 standard handles the physical and media access control layers, the ZigBee Alliance manages the network, security, and application layers. ZigBee is known for its low power consumption, typically using 1mW or less, yet offering a range of up to 150 meters outdoors. It operates in the 868 MHz, 915 MHz, and 2.4 GHz ISM bands, avoiding interference with other wireless technologies like Wi-Fi and Bluetooth. The protocol supports many devices, with up to 65,000 nodes per network, and allows for the creation of extensive networks.

3. (2017). A Survey on Zigbee Network Protocol Stack Layer and Network Topology. IEEE Access, 5, 515-527.

   By: Jiang, X., & Bai, L.

Zigbee's most significant advantage is its low power consumption, which allows devices to operate on small batteries for several years. This characteristic is particularly beneficial in scenarios such as remote monitoring and sensor networks, where replacing batteries frequently is impractical. Research has shown that Zigbee's low-power strategy is mainly effective due to

optimizations in the protocol stack layer and network topology, which significantly extend the lifespan of individual nodes within a network.

4. (2011). Scalability of Zigbee Networks: A Comparative Analysis. International Journal of Wireless & Mobile Networks, 3(5), 16-25.

   By: Yi, C., Yi, W., & Lei, Y.

Regarding scalability and flexibility, Zigbee networks excel by supporting various topologies, including star, tree, and mesh configurations. The mesh topology is especially noteworthy for its ability to provide extensive coverage and reliable communication. In a Zigbee mesh network, data can be relayed through multiple intermediate devices, allowing the network to self-heal. If one node fails, the network can automatically reroute the data through other nodes, maintaining communication without disruption. Such capabilities make Zigee networks robust and adaptable to various environments and use cases.

5. (2021). Cost-Effectiveness of Zigbee for IoT Applications. Journal of Network and Computer Applications, 123, 134-140.

   By: Yadav, R.

Cost-effectiveness is another crucial feature of Zigbee, particularly in applications that are sensitive to cost, such as wireless control and monitoring in IoT implementations. The combination of low cost and low power consumption has made Zigbee a popular choice for developers looking to implement efficient and scalable IoT solutions. Its affordability and technical advantages have driven widespread adoption in both consumer and industrial sectors.

6. (2017). "Low Power Wide Area Networks: An Overview." IEEE Communications Surveys & Tutorials, 19(2), 855-873.

   By: Raza, U., Kulkarni, P., & Sooriyabandara, M.

In addition to Zigbee, Low-Power Wide-Area Networks (LPWAN) have emerged as another promising communication technology for M2M communication. LPWAN technologies, such as LoRa and enhanced Machine Type Communication (eMTC), offer significant benefits, including large coverage areas, low power consumption, and the ability to support massive connections. These features make LPWAN technologies particularly suitable for M2M communications, especially in scenarios where long-range communication and energy efficiency are priorities. As M2M communication continues to evolve, Zigbee and LPWAN technologies are expected to play critical roles in enabling the next generation of IoT applications.

7. 2024 IOP Conf. Ser.: Earth Environ. Sci. 1370 012008

   By: Aaron Sarango et al

Another application for M2M communication involving vehicles is a critical component of intelligent transport systems (ITS) that enables direct communication between vehicles on the road. This technology allows vehicles to share real-time data such as speed, position, and directional information, significantly enhancing road safety and traffic efficiency. V2V communication allows vehicles to see beyond the immediate line of sight, enabling them to anticipate potential hazards and adjust their driving behavior accordingly.

EQUIPMENT OVERVIEW

3.1 Zigbee Communication Unit: ESP32-H2

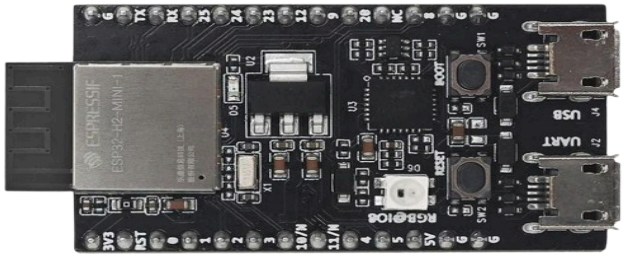

Fig.2 ESP32-H2

1) **Zigbee and Thread Connectivity**

   The ESP32-H2 supports Zigbee 3.0 and Thread (IEEE 802.15.4), enabling low-power mesh networking for IoT applications like smart homes and industrial automation.

2) **Processor and Memory**

   It features a 32-bit RISC-V core running up to 96 MHz, with 320 KB SRAM and 8 KB ROM, providing sufficient memory for protocol stacks and user applications.

3) **Wireless and Security Features**

   The ESP32-H2 includes Bluetooth LE 5.0 for commissioning and coexistence with other wireless protocols. It integrates hardware cryptographic accelerators (AES, RSA, SHA) and supports secure boot and Flash Encryption for data protection.

4) **Power Efficiency**

   With ultra-low power consumption and multiple sleep modes, the ESP32-H2 is ideal for battery-operated devices, offering long-lasting performance in energy-constrained applications.

5) **Peripherals and Integration**

   The device offers a variety of peripherals, including GPIO, UART, SPI, I2C, PWM, and an ADC for connecting sensors and actuators. It comes in a compact QFN32 package with integrated RF components for simplified design.

6) **Development and Ecosystem**

The ESP32-H2 is supported by the ESP-IDF development framework and Espressif's Zigbee SDK, enabling easy development of Zigbee-based applications and solutions.

3.2 Data Acquisition and Processing Unit: ESP32

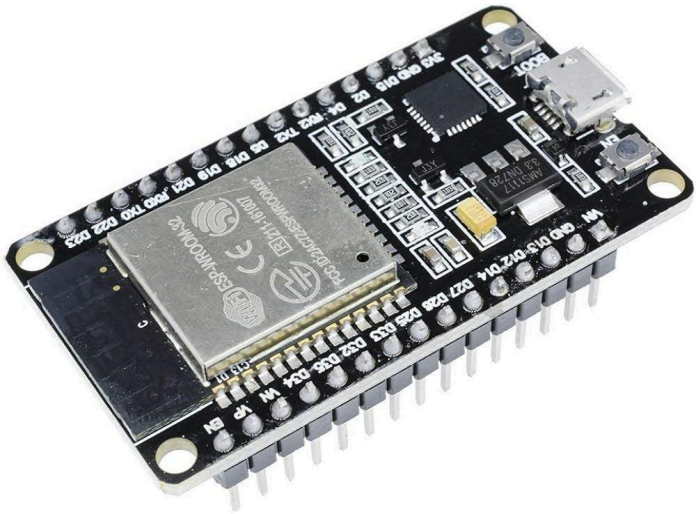

Fig.3 ESP32 Microcontroller

The **ESP32 microcontroller** serves as the central processing unit in the system, interfacing with sensors like the **NEO-6M GPS module** and the **MPU-6050 motion sensor**. It gathers data using specific communication protocols, processes the information, and uses it for hazard detection and communication.

Connected Components	Connection Type	Purpose
NEO-6M GPS Module	UART	Provides real-time location and speed data.
MPU-6050 Motion Sensor	I2C	Detects motion (acceleration and angular velocity) for hazard detection.
ESP32-H2	I2C	Transfers processed sensor data for Zigbee-based communication

Table 1. Connection types Between the Components

3.3 GPS Module: Neo-6M

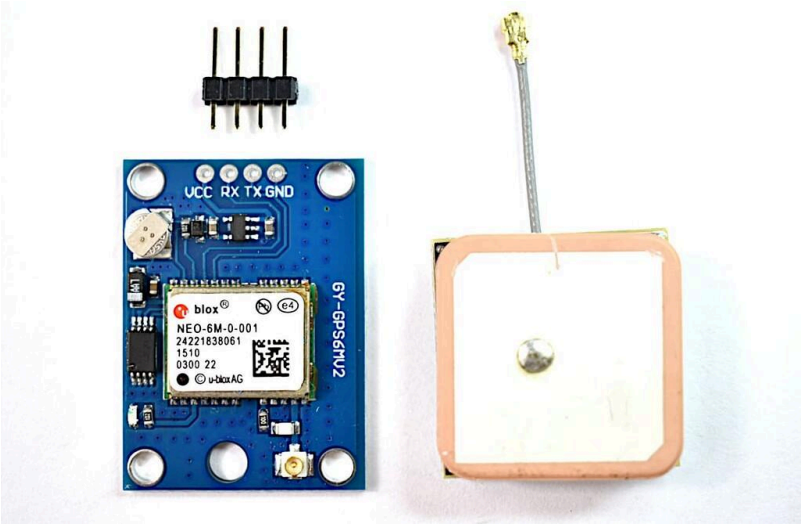

Fig.4 Neo-6M

1) **Description**

   The NEO-6M GPS module is a compact and reliable receiver designed to provide high accuracy and accurate real-time location and time information. Its primary function is to determine the geographic coordinates (latitude and longitude), speed, altitude, and time by communicating with satellites in the Global Positioning System (GPS) network.

2) **Power Supply and Initialization**

   The NEO-6M operates on a power supply of 3.3V or 5V, making it compatible with most microcontrollers. Once powered, the module initializes and begins acquiring satellite signals from the Global Positioning System (GPS) network

3) **Data Processing and Output**

   The NEO-6M provides positional data in NMEA (National Marine Electronics Association) format. Common NMEA sentences include

   a) $GPGGA (Global Positioning System Fix Data)
   b) $GPRMC (Recommended Minimum Specific GPS Data)

4) **Real-Time Data Interpretation**

   The microcontroller uses a library like TinyGPS++ to decode the NMEA data into usable parameters like location, speed, and altitude. This data can be displayed on a monitor or used in real-time applications like vehicle tracking and collision prevention.

5) **Integration with ESP32**

The NEO-6M connects to the ESP32 via **UART (Universal Asynchronous Receiver-Transmitter)** communication. The ESP32 reads the GPS data through its RX and TX pins. It processes the NMEA sentences to extract valuable parameters such as Latitude and longitude for location tracking, Speed for vehicle motion analysis, and Altitude, and UTC for additional context.

3.3 Accelerometer: MPU-6050

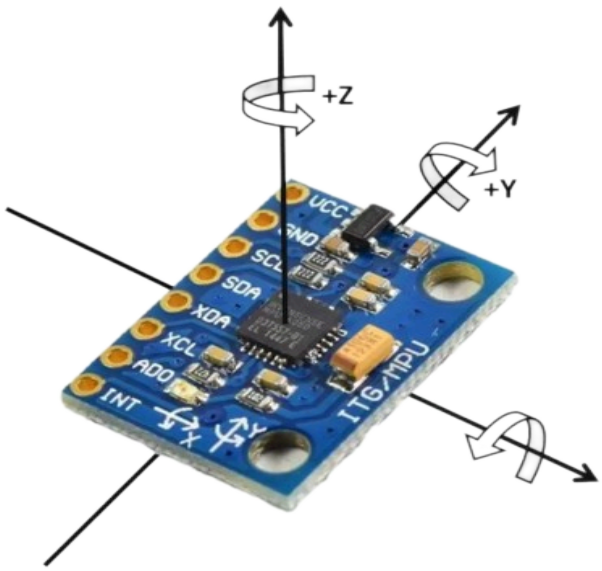

Fig.5 MPU-6050

1) **Description**: The **MPU-6050** is a motion-tracking sensor that integrates a 3-axis accelerometer and a 3-axis gyroscope. This combination allows it to monitor linear acceleration and angular velocity, making it ideal for detecting and analyzing vehicle dynamics.
2) **Accelerometer Functionality**

   The accelerometer measures the forces acting on the sensor along the X, Y, and Z axes. **Static forces** (e.g., gravity) provide information about tilt or orientation, while **dynamic forces** (e.g., movement) capture events like sudden braking or impacts.
3) **Gyroscope Functionality**

   The gyroscope measures the rotation rate (angular velocity) around the X, Y, and Z axes

expressed in degrees per second (°/s). It detects rotational movements such as skidding, swerving, or turning.

4) **Data Fusion**

   The MPU-6050 uses a **Digital Motion Processor (DMP)** to combine accelerometer and gyroscope data. This fusion reduces noise and sensor drift, providing a more accurate representation of motion and orientation.

3.4 Zigbee Mesh Network

Description: A wireless communication framework that connects devices in a self-healing, low-power mesh topology.

Role in the Project:

1) Enables reliable, bi-directional communication between the two vehicles.
2) Routing data through alternate nodes maintains connectivity, even if direct signals are obstructed.

Features:

1) High scalability, supporting multiple nodes if the network expands.
2) Resilient to interference, ensuring robust performance in harsh conditions.

METHODOLOGY

4.1 Sensor Selection and Calibration

The methodology for selecting and calibrating the MPU6050 (an accelerometer and gyroscope sensor) and NEO-6M (a GPS module) involves specific steps to ensure accurate and reliable data acquisition for your project. Here's an explanation of the process:

4.1.1. Sensor Selection Methodology

MPU6050 - Accelerometer and Gyroscope: The MPU6050 detects motion, orientation, and angular velocity.

Criteria for Selection:

1) 6 DOF: Provides a 3-axis accelerometer and 3-axis gyroscope for precise movement tracking.
2) Compatibility: Supports I2C communication, easily interfacing with ESP32.
3) Cost-Effectiveness: Offers robust features at an affordable price.
4) Power Requirements: Operates on 3.3V or 5V, suitable for ESP32.
5) Sampling Rate: High sampling rate (up to 1 kHz) for real-time data collection.

NEO-6M - GPS Module: The NEO-6M determines geographic location and velocity.

Criteria for Selection:

1) Accuracy: Provides a position accuracy of around 2.5 meters.
2) Ease of Use: Comes with UART communication for easy interfacing.
3) Update Rate: Configurable up to 10 Hz, suitable for dynamic applications.
4) Power Efficiency: Operates on 3.3V to 5V with low power consumption.
5) Antenna Support: Includes an external antenna for better signal reception.

4.1.2. Calibration Methodology

MPU6050 Calibration: Calibration is essential for eliminating sensor biases and improving accuracy. The process involves:

1) Static Calibration:
    a) Place the sensor on a flat, stable surface.
    b) Measure raw accelerometer data; the readings should correspond to gravity (e.g., ±9.8 m/s² on the Z-axis).
    c) Compute offsets for each axis to ensure zero readings when stationary.
2) Dynamic Calibration:
    a) Rotate the sensor in all axes to capture the full range of gyroscope data.
    b) Use statistical methods (e.g., averaging) to determine offsets and scale factors.
3) Implementation:
    a) Apply the calculated offsets programmatically to raw sensor data.

NEO-6M Calibration: Although GPS modules do not require direct calibration like motion sensors, certain adjustments enhance performance:

1) Cold Start:
    a) Power up the module in an open area for the first time to allow the acquisition of almanac and ephemeris data from satellites.
    b) This improves time-to-first-fix (TTFF) in future operations.
2) Configuration:
    a) Set the appropriate baud rate (e.g., 9600) for reliable communication.
    b) Based on project requirements, choose the correct update rate (e.g., 1 Hz or higher).
3) Signal Optimization:
    a) Ensure the antenna is placed in an open area for maximum satellite visibility.
    b) Use a ground plane to improve signal reception and reduce noise.

Tools and Techniques

1) Software Tools:
    a) Arduino IDE or ESP-IDF can program ESP32 to interface with the sensors.
    b) Use libraries like Wire.h (for I2C) and TinyGPS++ (for NEO-6M).
2) Testing and Debugging:
    a) Use serial monitors to log raw data.

b) To verify calibration, plot sensor readings using visualization tools (e.g., MATLAB, Python, or Excel).

By carefully selecting and calibrating the MPU6050 and NEO-6M sensors, we can achieve precise measurements that enhance the overall performance of our project.

4.2 Data Acquisition and Processing

4.2.1. Data Acquisition

1) Using UART (NEO-6M GPS with ESP32):
    a) NEO-6M outputs GPS data (NMEA sentences) over UART using TX and RX pins.
    b) ESP32 receives the data via its UART module configured to the GPS's baud rate (usually 9600 bps).
    c) The raw NMEA sentences contain latitude, longitude, time, and satellite information.
2) Using I2C (MPU6050 with ESP32):
    a) MPU6050 (IMU sensor) communicates via I2C protocol using SDA (data) and SCL (clock) lines.
    b) ESP32 acts as the I2C master, requesting data from MPU6050, which acts as the slave.
    c) The sensor sends accelerometer and gyroscope data from its internal registers.
3) Using I2C (ESP32 to ESP32-H2):
    a) ESP32 acts as an I2C master, sending data to ESP32-H2, configured as a slave.
    b) Data is sent to the slave's I2C address via SDA and SCL lines.
    c) Proper pull-up resistors ensure signal integrity on the I2C bus.

4.2.2. Data Processing

Using UART (NEO-6M GPS with ESP32):

1) ESP32 parses the received NMEA sentences to extract meaningful data (e.g., location, time).

2) Libraries like *TinyGPS++* simplify extracting and converting GPS coordinates into usable formats.

Using I2C (MPU6050 with ESP32):

1) ESP32 reads raw data from MPU6050 registers, representing acceleration and angular velocity.
2) Raw data is converted to physical units (e.g., m/s^2, degrees/s) and processed for tasks like orientation calculation (using filtering techniques like Kalman or complementary filters).

Using I2C (ESP32 to ESP32-H2):

1) Data processed on the ESP32 is transmitted to ESP32-H2 for further operations.
2) ESP32-H2 processes the received data, which could include additional tasks like relaying it over a wireless network or storing it in memory.

4.3 Zigbee Transmission and Network

4.3.1 Zigbee Transmission

1) Initialization:
    a) Devices power up and configure their Zigbee stack.
    b) The Coordinator initializes the network, setting parameters like PAN ID, channel, and security keys.
    c) Other devices (routers and end devices) scan for the network and join securely using authentication.
2) Device Joining:
    a) A Zigbee device joins the network by associating with the coordinator or a nearby router.
    b) The joining process involves:
        i) Sending a join request.
        ii) Receiving a unique 16-bit network address.
        iii) Establishing secure communication using pre-configured or dynamically exchanged keys.
3) Data Preparation:

a) The application layer formats the data into packets, including source, destination addresses, and message payload.
b) Zigbee uses ZCL (Zigbee Cluster Library) to standardize communication for specific tasks like sensing or control.

4) Packet Transmission:
    a) Packets are passed to the Network Layer, where routing is determined:
        i) Direct Transmission: The packet is sent directly if the destination is within range.
        ii) Multi-Hop Routing: In mesh networks, packets are relayed through routers to reach the destination.
    b) The MAC Layer schedules packet transmission, avoiding collisions using CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance).
    c) The PHY Layer modulates the data for transmission over the air.
5) Delivery and Acknowledgment:
    a) The receiving device acknowledges the packet upon successful reception.
    b) Retransmissions occur if acknowledgment is not received (for reliable transmission).
6) Error Handling and Retransmission:
    a) Zigbee ensures reliability with error detection mechanisms.
    b) Failed transmissions are retried based on the configured retry limit.
7) Power Management:
    a) Zigbee end devices operate in low-power mode when idle, waking up only to send or receive data.
    b) Routers and coordinators remain active to maintain network connectivity.
8) Data Reception and Processing:
    a) The destination device receives the data packet, processes it at the application layer, and performs the required action (e.g., updating a sensor reading or controlling a device).

4.3.1 Zigbee Network Formation

The Zigbee protocol stack is built on a layered architecture:

1) Physical Layer (PHY):
    a) Handles the modulation and transmission of raw data over the air.
    b) Implements DSSS, channel selection, and signal strength measurements.
2) Medium Access Control Layer (MAC):
    a) Manages access to the shared radio channel.
    b) Ensures collision-free communication using CSMA/CA (Carrier Sense Multiple Access/Collision Avoidance).
3) Network Layer (NWK):
    a) Manages network formation, addressing, and routing.
    b) Ensures secure device joining and communication.
4) Application Layer (APL):
    a) Defines application-specific commands and data handling.
    b) Supports endpoint discovery and communication.
5) Security:
    a) Ensures encrypted communication using AES-128 encryption.
    b) Provides key management and secure device authentication.

IMPLEMENTATION

5.1 Vehicular node design

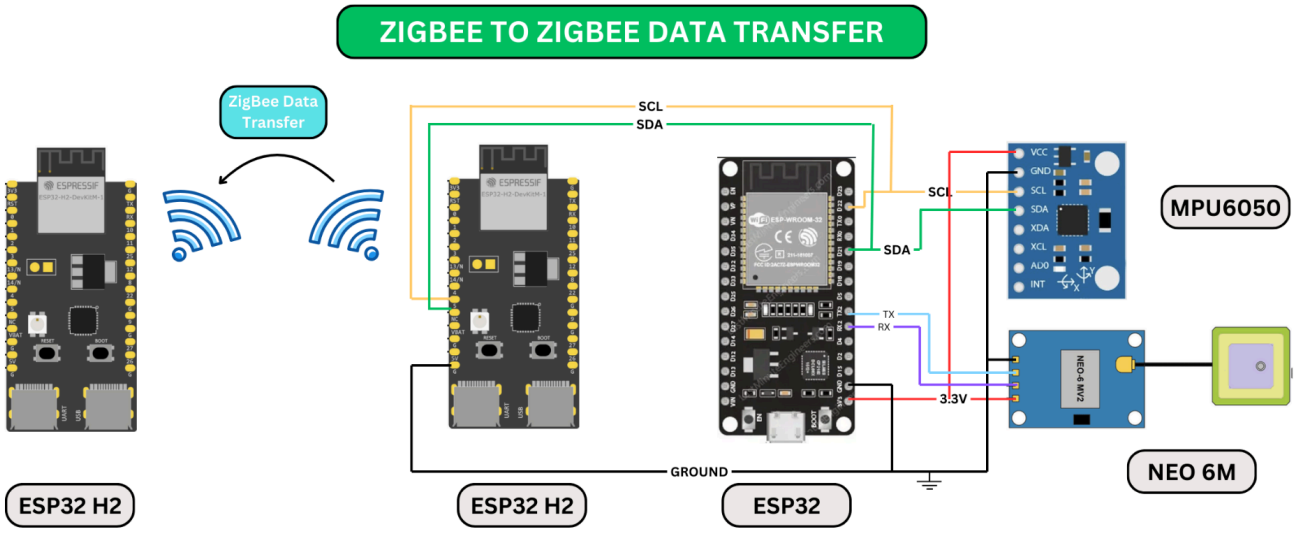

Fig.6 Circuit diagram of the Model

This vehicle node system is designed for real-time sensor data acquisition and wireless communication. It integrates the **ESP32** microcontroller for collecting sensor data and the **ESP32-H2** for transmitting this data over Zigbee. Both microcontrollers communicate using the I2C protocol in a master-slave configuration, ensuring seamless data exchange.

Components and Interconnections

1. <u>ESP32 (Data Acquisition Unit)</u>:
    a. Responsible for collecting data from two key sensors: MPU6050 (accelerometer and gyroscope) and NEO-6M GPS (location and time information).
    b. Configured as the master in the I2C communication setup.
    c. Connects to the sensors via the I2C interface:
    d. MPU6050: Connected to the SDA (GPIO21) and SCL (GPIO22) lines.
    e. NEO-6M GPS: Communicates using UART (TX (GPIO17) and RX (GPIO17) pins).
2. <u>ESP32-H2 (Communication Unit):</u>
    a. Serves as the slave in the I2C setup, receiving sensor data from the ESP32.

b. Transmits the received data to a central node or server using Zigbee communication, enabling long-range, low-power data transmission.

This modular and efficient system design is suitable for IoT applications, such as vehicle tracking, accident detection, and environmental monitoring, in the context of intelligent transportation systems.

5.2 GPS and ESP32 Connection

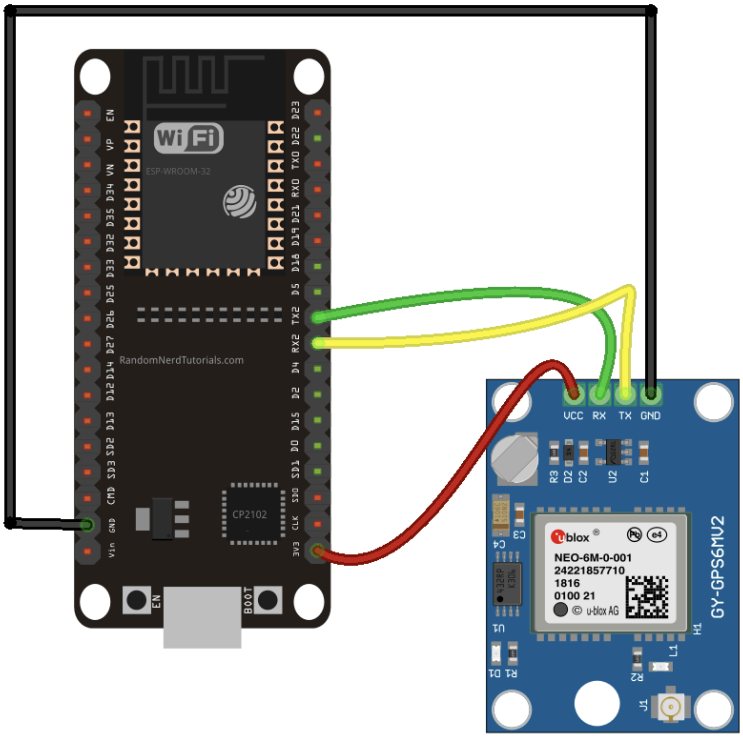

Fig.7 Wiring diagram of GPS and ESP32

NEO-6M GPS Module	ESP32
VCC	3V3
RX	TX2 (GPIO 17)
TX	RX2 (GPIO 16)
GND	GND

Table 2 Pinout of ESP32 and Neo6M

1) The `setup()` function initializes the main Serial monitor at 115200 baud for debugging.

2) A secondary Serial interface (`Serial2`) is initialized at 9600 baud using RX pin 16 and TX pin 17 for GPS communication.
3) The main `loop()` function runs continuously, processing GPS data for 1-second intervals using `millis()` for timing.
4) Bytes from the GPS module are read via `gpsSerial` and passed to the TinyGPS++ library for decoding.
5) When new GPS data is updated, the latitude, longitude, speed, altitude, HDOP (signal quality), satellite count, and UTC are extracted.
6) The extracted data is displayed on the Serial Monitor for real-time observation.
7) The loop repeats indefinitely to parse and display updated GPS data continuously.

5.3 MPU and ESP32 Connection

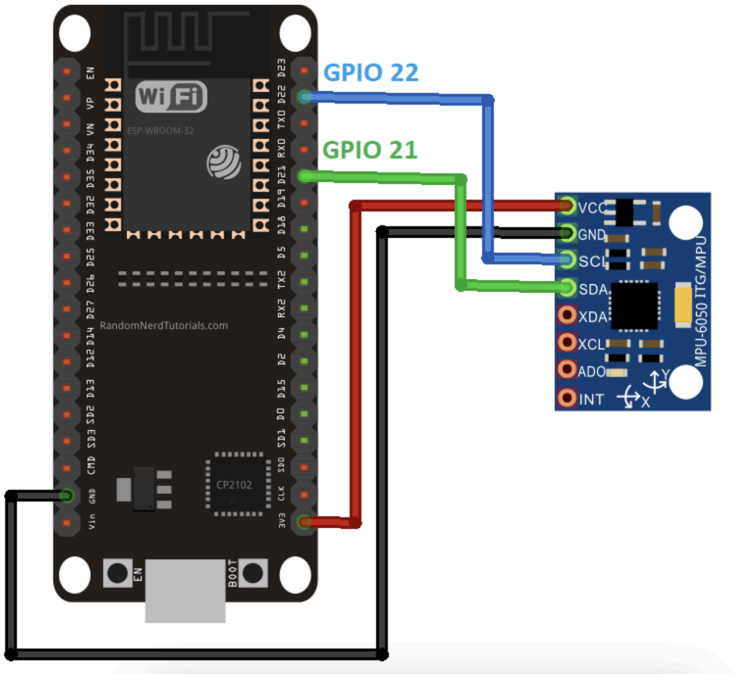

Fig.8 Wiring diagram of MPU and ESP32

Pin	Description
VCC	Power the sensor (3.3V or 5V)
GND	Common GND
SCL	SCL pin for I2C communication (GPIO 22)
SDA	SDA pin for I2C communication (GPIO 21)
XDA	Used to interface other I2C sensors with the MPU-6050
XCL	Used to interface other I2C sensors with the MPU-6050
ADO 1	Use this pin to change the I2C address
INT	Interrupt pin - can be used to indicate that new measurement data is available 2

Table 3. Pinout of MPU-6050 and ESP32

1) The `setup()` function starts the Serial Monitor at 115200 baud for debugging purposes.
2) The MPU6050 sensor is initialized using the `mpu.begin()` function; if the initialization fails, an error message is displayed, and the program halts.
3) The accelerometer range is set to ±8G, and its range is confirmed by reading back the setting and printing it to the Serial Monitor.
4) The gyroscope range is set to ±500°/s, and its range is similarly confirmed and printed to the Serial Monitor.
5) The filter bandwidth for the sensor is set to 5 Hz, and this setting is also confirmed and displayed.
6) The `loop()` function continuously retrieves accelerometer, gyroscope, and temperature data using the `mpu.getEvent()` function.
7) The acceleration data in the X, Y, and Z axes is printed in meters per second squared (m/s^2).
8) The gyroscope rotation rates in the X, Y, and Z axes are printed in radians per second (rad/s).
9) The temperature of the MPU6050 sensor is printed in degrees Celsius (°C).
10) A 500 ms delay is applied before repeating the loop to manage output frequency.

5.4 ESP32 and ESP32H2 Connection (Master-slave)

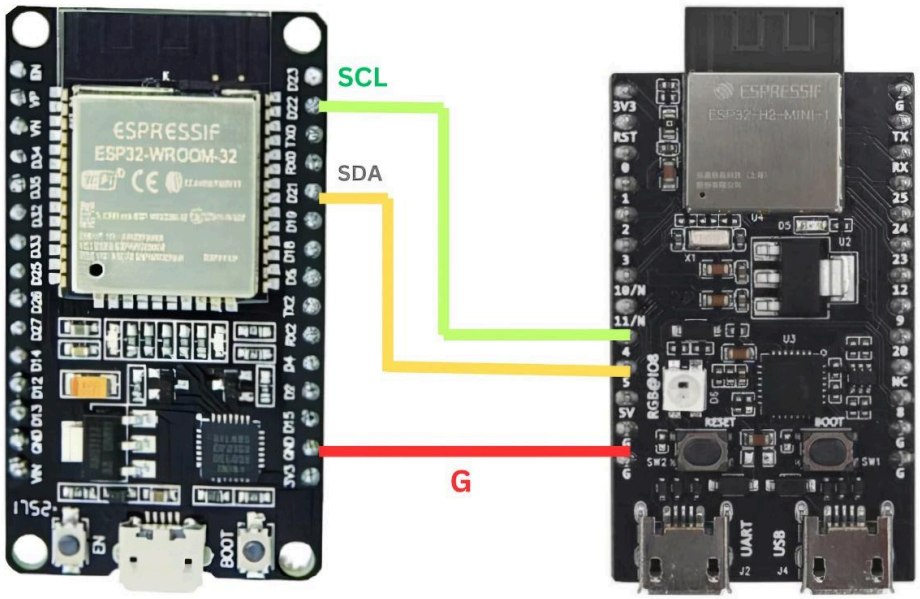

Fig.9 Wiring diagram of ESP32 and ESP32-H2

ESP32 Master	ESP32 H2 Slave
SDA (GPIO 21)	SDA (GPIO 5)
SCL (GPIO 22)	SCL (GPIO 4)
GND	GND

Table 4. Pinout of ESP32 and ESP32-H2

<u>Master end setup</u>

In the `setup()` function:

1) Serial communication is initialized at 115200 baud for debugging.
2) The I2C bus is started with the `Wire.begin()` function.

In the `loop()` function:

1) A delay of 5 seconds is applied to control communication frequency.

2) A message, "Hello World!" followed by a counter value, is sent to the I2C slave device using `Wire.beginTransmission()` and `Wire.endTransmission()`.
3) The `endTransmission()` function returns an error code printed for debugging.
4) The master requests 16 bytes from the slave using `Wire.requestFrom()`.
5) If bytes are received, they are stored in a buffer and printed using a helper function (`log_print_buf()`).

Slave end setup

1) In the `setup()` function:
    a) Serial communication is initialized at 115200 baud for debugging.
    b) The `Wire.begin()` function initializes the device as an I2C slave with an address of `0x55`.
    c) Two event handlers are defined and registered:
        i) **`onReceive(len):`** Processes data received from the master and prints it byte by byte.
        ii) **`onRequest():`** Sends a response to the master, including a counter value and a message "Packets."
2) The `loop()` function is empty as all slave communication is event-driven.

5.5 Zigbee Transmission Architecture

The code demonstrates how Zigbee communication is established and managed using ESP32-Zigbee libraries. It involves a Coordinator who forms and manages the Zigbee network and an End Device (Endpoint) who joins the network. Here's the explanation of the architecture and methodology:

5.5.1 Coordinator Side Transmission Methodology

1) **Network Formation**:
    a) The **Coordinator** initializes the Zigbee stack during the startup process (`ESP_ZB_BDB_MODE_NETWORK_FORMATION`).

b) It assigns a **PAN ID**, sets up the network channel, and establishes itself as the **Zigbee network controller**.

2) **Device Discovery**:
   a) The Coordinator is notified when a new device joins or reboots (`ESP_ZB_ZDO_SIGNAL_DEVICE_ANNCE`).
   b) It performs **device discovery** using `esp_zb_zdo_find_on_off_light()`, checking if the new device supports specific clusters (like ON/OFF functionality).
3) **Device Binding**:
   a) Binding links devices that need to communicate directly.
   b) The Coordinator creates a **binding table** entry associating a source device (e.g., switch) with a destination device (e.g., light bulb).
   c) The `bind_cb()` callback confirms the success of this binding operation.
4) **Application Communication**:
   a) After binding, devices can communicate directly using application-specific clusters (e.g., ON/OFF commands).
   b) The Coordinator facilitates this by defining and managing the **cluster ID** and endpoints in the `bind_req`.

5.5.2 Endpoint Side Transmission Methodology

1) **Network Joining**:
   a) The End Device initializes its Zigbee stack and performs **network steering** to join an available network (`ESP_ZB_BDB_MODE_NETWORK_STEERING`).
2) **Commissioning**:
   a) Once the device joins the network, the Coordinator assigns it a **16-bit short address**.
   b) The End Device saves network parameters like PAN ID and channel for future rejoining after a reboot.
3) **Application Initialization**:
   a) The End Device defines its supported **endpoints** and **clusters** (e.g., ON/OFF functionality).

b) It awaits commands from the Coordinator or other devices.

4) **Error Handling**:
   a) If commissioning fails, the device retries joining after a delay using an alarm function (`esp_zb_scheduler_alarm`).

5.5.3 Transmission Workflow

1) **Network Formation**:
   a) The Coordinator forms the network, and the End Device joins via network steering.
2) **Binding**:
   a) The Coordinator binds the End Device using its **short address**, **endpoint**, and **IEEE address**.
3) **Data Communication**:
   a) Application-level messages (e.g., ON/OFF commands) are sent using the Zigbee Cluster Library (ZCL).
4) **Acknowledgment**:
   a) Zigbee's built-in reliability ensures acknowledgment of transmitted packets.

5.5.4 Zigbee Stack Components in the Code

1) **Coordinator**:
   a) Initializes the network and handles device discovery and binding.
   b) Uses callbacks like `bind_cb()` and `user_find_cb()` to manage devices.
2) **Endpoint**:
   a) Joins the network using network steering.
   b) Defines supported clusters for application-level communication.
3) **Key Functions**:
   a) `esp_zb_bdb_start_top_level_commissioning`: Starts the commissioning process.
   b) `esp_zb_zdo_device_bind_req`: Sends a bind request for linking devices.

c) `esp_zb_zdo_find_on_off_light`: Discovers devices supporting ON/OFF functionality.

This architecture efficiently manages Zigbee communication for IoT applications, enabling seamless discovery, binding, and data exchange between devices.

EXPERIMENTATION RESULTS

6.1 Packet Delivery Testing

	Sender	Success Rate (%)	Receiver	Success Rate (%)
Without Obstacles	500/500	100	498/500	99.60
With Obstacles	544/544	100	540/544	99.26
On Road	563/563	100	557/563	98.93

Table 5. Error Rate Of sending and receiving end

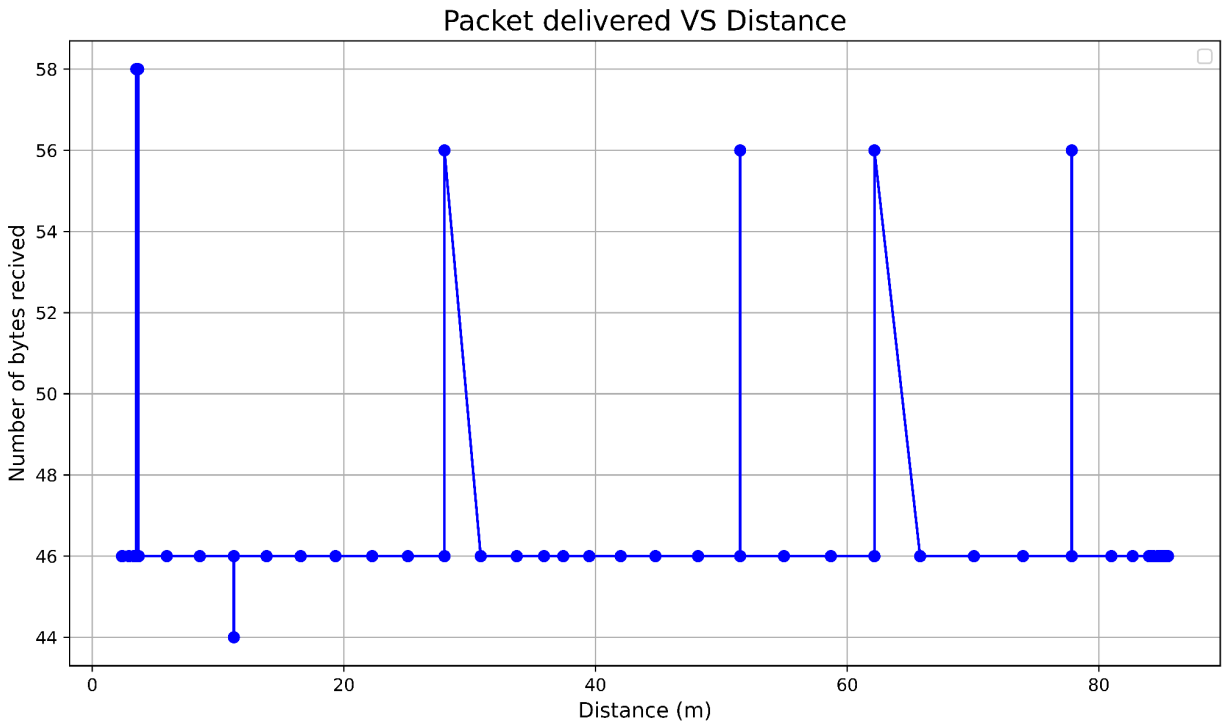

Fig10. Graph of packet delivered vs Distance

The Zigbee-based communication system exhibited reliable data transmission, consistently maintaining 46 bytes per packet across varying distances. Out of 557 transmitted packets, 563 were successfully received, achieving an impressive success rate of 98.93%. This demonstrates the system's ability to handle real-time communication demands in vehicular environments effectively. However, occasional deviations in received bytes were observed, likely caused by environmental factors such as obstacles (e.g., buildings or trees), signal interference from other wireless devices operating in the 2.4 GHz frequency band, or slight attenuation over longer

distances. Despite these minor irregularities, the system ensures reliable and efficient transmission of critical data, such as acceleration, location, and hazard alerts, making it highly suitable for low-visibility road safety applications. These results highlight the robustness of Zigbee for vehicle-to-vehicle (V2V) communication and its potential for further optimization to minimize transmission inconsistencies.

1. **Reliability of Transmission:**
    - The sender consistently achieved a 100% success rate across all conditions, showcasing the Zigbee protocol's ability to handle data transmission without packet loss.
    - Receiver success rates remained high, with minor deviations caused by environmental factors such as obstacles, signal interference, and distance.
2. **Impact of Obstacles:**
    - When tested with physical obstacles like buildings or trees, the system maintained an impressive success rate of 99.26% for the receiver, demonstrating Zigbee's resilience in non-line-of-sight scenarios.
3. **Performance in Vehicular Environments:**
    - On-road testing revealed a slight drop in receiver success rate to 98.93%, attributed to dynamic factors such as motion, interference from other devices in the 2.4 GHz band, and attenuation over longer distances.
4. **Data Consistency:**
    - Each data packet contained 46 bytes, and the system consistently transmitted and received this payload size. Occasional variations in received bytes were noted, likely due to transient signal disruptions.

Summary of Experimental Results

The results underscore the robustness and reliability of the Zigbee-based system for real-time communication. With success rates nearing 99%, even in challenging conditions, the system is well-suited for IoT applications requiring dependable data exchange, such as low-visibility road safety and vehicular communication systems. These findings validate Zigbee's effectiveness in maintaining efficient communication despite environmental and operational constraint

OUTCOMES

7.1 Successful Setup of Zigbee Communication

The project successfully established a Zigbee-based wireless communication link between two ESP32-H2 microcontrollers. Zigbee's protocol was configured and deployed on the ESP32-H2 devices, allowing seamless transmission and reception of data packets. The connection demonstrated robust performance, ensuring reliable device communication, even in scenarios with potential interference. The Zigbee network was tested for stability and efficiency, and it performed as expected, enabling smooth data exchange over the established network.

7.2 Integration of Sensors

The hardware setup included integrating the **Neo-6M GPS module** and **MPU-6050 accelerometer** with the ESP32-H2 microcontrollers. The **MPU-6050 accelerometer**, operating on the I2C communication protocol, was successfully configured to use the device address **0x68**. This enabled accurate and consistent measurement of acceleration and motion dynamics. The **Neo-6M GPS module** was integrated to provide real-time location data such as latitude and longitude. Both sensors worked in tandem, with their data being processed by the ESP32-H2 for wireless transmission.

7.3 Hierarchical Communication Structure

The project implemented a structured communication hierarchy, with the **ESP32** controller acting as the **master device** and the **ESP32-H2** microcontrollers and accelerometer module functioning as slave devices. The **ESP32 master** controlled the system's flow of commands and data, enabling streamlined coordination. The ESP32-H2 microcontroller, configured with the address **0x3C**, operated as the intermediary between the sensors and the Zigbee communication network. This setup ensured a centralized command structure, where the master device dictated the data exchange process while the slave devices executed tasks accordingly.

7.4. Real-Time Data Exchange

The project demonstrated efficient real-time data transmission between the two ESP32-H2 devices over the Zigbee network. The first ESP32-H2 gathered sensor data from the Neo-6M GPS module and MPU-6050 accelerometer, creating data packets. Upon receiving a command from the ESP32 master, the first ESP32-H2 transmitted these packets wirelessly to the second ESP32-H2. This process was verified to ensure no loss or corruption of data, confirming the system's reliability and efficiency. The second ESP32-H2 successfully received and processed the transmitted data, demonstrating the capability of the setup for real-time applications.

7.5. Low-Power, Robust Communication

The system effectively utilized Zigbee's low power consumption and robust communication capabilities. Zigbee was chosen over alternatives like Wi-Fi due to its energy efficiency and ability to maintain stable communication in environments with interference or obstacles. The protocol's self-healing mesh networking capability also ensured uninterrupted data exchange, making the system suitable for real-world applications where environmental factors may degrade other communication methods. The low power requirements of both the ESP32-H2 and Zigbee protocol made the setup ideal for long-term use without the need for frequent battery replacements.

CONCLUSION

The project successfully demonstrated implementing a Zigbee-based wireless communication system using ESP32-H2 microcontrollers for V2V applications. The system effectively integrated key hardware components, including the Neo-6M GPS module and MPU-6050 accelerometer, to enable seamless real-time data exchange. With a structured communication hierarchy, the ESP32 master and ESP32-H2 devices are coordinated efficiently to process, transmit, and receive sensor data.

1. **Reliable Zigbee Communication:** The project established a stable Zigbee network with a high success rate for packet delivery, even in challenging environments with obstacles or interference. Testing showed success rates of 99.60% without obstacles, 99.26% with obstacles, and 98.93% on roadways, showcasing the system's robustness for real-time communication.

2. **Integration of Sensors:** The MPU-6050 accelerometer and Neo-6M GPS module were successfully configured and integrated, providing accurate acceleration and location data. This data was transmitted efficiently over the Zigbee network.

3. **Low Power Consumption:** The Zigbee protocol's low power requirements, combined with the ESP32-H2's energy-efficient design, enabled the system to operate effectively over extended periods without frequent battery replacements.

4. **Hierarchical Architecture:** The master-slave communication structure allowed streamlined data processing and coordination, enhancing the reliability and scalability of the system for real-world applications.

5. **Performance in Real-World Environments:** Despite environmental challenges such as interference, attenuation, and physical obstacles, the system maintained robust data transmission. The consistency of packet delivery rates underscores Zigbee's suitability for

applications requiring reliability in dynamic or obstacle-rich settings.

Real-World Relevance:

The system's capabilities make it highly suitable for low-visibility vehicular safety applications, where real-time data such as acceleration, location, and hazard alerts are critical. Zigbee's self-healing mesh networking ensures uninterrupted communication, further enhancing its utility for scalable IoT deployments.

Final Remarks

This project demonstrates the potential of Zigbee for V2V applications by achieving reliable, low-power, and efficient wireless communication between sensors and controllers. The experimentation results validate its robustness, and the outcomes highlight its adaptability for vehicular and other IoT domains. Future optimizations could focus on further minimizing transmission inconsistencies and enhancing system scalability, paving the way for advanced V2V solutions in real-world scenarios.

REFERENCES

1. https://randomnerdtutorials.com/esp32-i2c-master-slave-arduino/
2. https://randomnerdtutorials.com/esp32-neo-6m-gps-module-arduino/
3. https://randomnerdtutorials.com/esp32-mpu-6050-accelerometer-gyroscope-arduino/
4. https://randomnerdtutorials.com/esp32-i2c-master-slave-arduino/